



AUTOMATISER VOS DEPLOIEMENTS FABRIC AVEC fabric-cicd

Merci à nos mécènes

Ils rendent l'événement possible.



[AUTOMATISER VOS DEPLOIEMENTS FABRIC AVEC fabric-cicd]



Axel SAMSON

Tech Lead

Datatorii

Points clés de la session

- Comprendre les différentes options de déploiement dans Fabric
- Focus sur la librairie fabric-cicd
- fabric-cicd en action

Bio

Consultant en data depuis 7 ans, principalement sur les technologies Microsoft et Databricks.

Fabric Microsoft certified • [LinkedIn](#) / [Dataticot](#)

[AUTOMATISER VOS DEPLOIEMENTS FABRIC AVEC fabric-cicd]



Pierrick NEYME

Tech Lead

Datatorii

Points clés de la session

- Comprendre les différentes options de déploiement dans Fabric
- Focus sur la librairie fabric-cicd
- fabric-cicd en action

Bio

Consultant en data depuis 14 ans, spécialisé dans la technologie Microsoft (SQL Server, Azure, Fabric).

Fabric Microsoft certified • [LinkedIn](#)

01

Les options de CI/CD dans Fabric

Aperçu et comparaison des principales options de CI/CD dans Microsoft Fabric



Rappels sur la CI/CD

Continuous Integration / Continuous Deployment

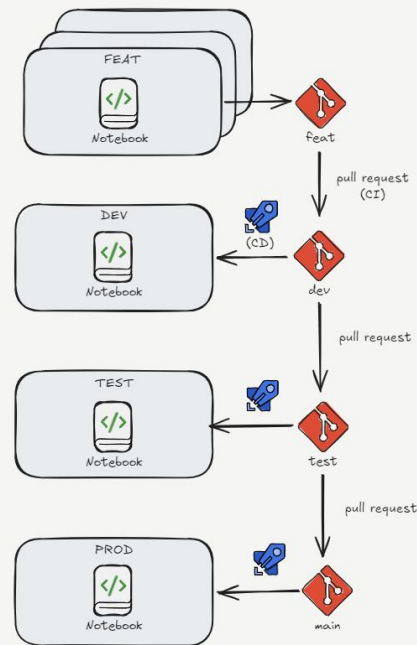
Ensemble de méthodes et d'outils facilitant l'intégration de code et son déploiement entre plusieurs environnements.

Continuous Integration

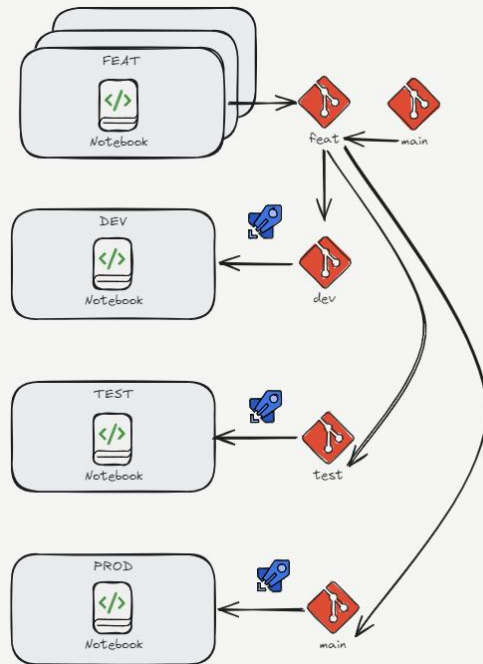
Exécution de tests automatisés (tests unitaires, tests de non-régression, vérification des bonnes pratiques, etc.).

Continuous Deployment

Déploiement effectif des artefacts du projet entre les différents environnements (DEV, TEST, PROD).



Workflows git & Process de développement



Les options de CI/CD dans Microsoft Fabric

Pipelines de déploiement Fabric

Pipelines low-code qui permettent de déployer le contenu d'un workspace à un autre via une interface graphique.

Déploiements basés sur git

Déploiements basés sur la synchronisation entre un workspace Fabric et une branche d'un repo git.

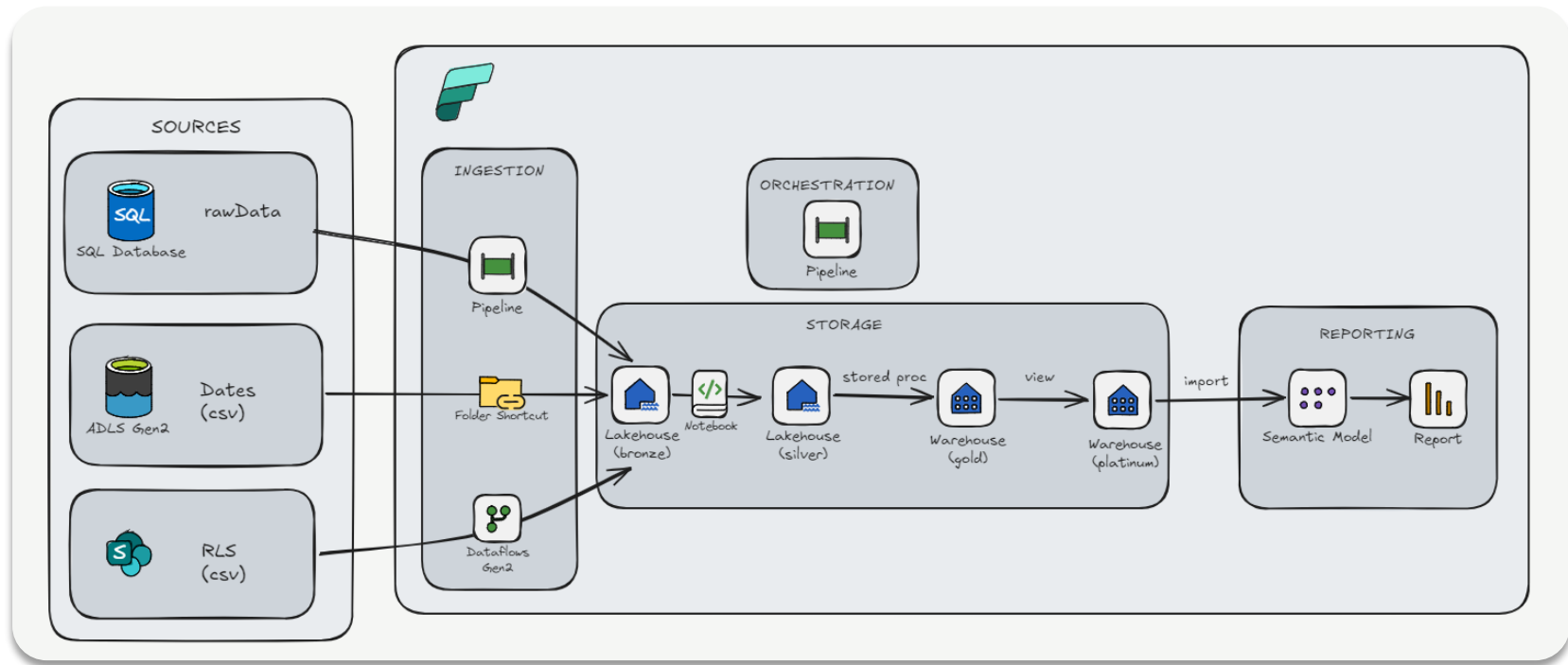
fabric-cicd

Déploiements effectués par la librairie Python fabric-cicd, wrapper autour des API REST de Fabric.



fabric-cicd

Environnement de la démo



DÉMO

Déploiement avec Pipelines de déploiement Fabric

Pipelines de déploiement Fabric

Avantages

- Simple à créer/utiliser (interface graphique)
- Permet le Selective Deployment
- Règles de déploiement pour certains types d'objet
- Historique des déploiements

Inconvénients

- Nécessite workspace Fabric ou PPU (pas de pro)
- Nécessite des permissions élevées sur les workspaces de source et de destination
- La mise en place des règles de déploiement nécessite que l'objet existe en destination → complexité du premier déploiement
- La consultation/modification des règles de déploiement nécessite d'être propriétaire de l'objet en source et destination
- Ne gère pas la suppression d'objets
- Pas de déclenchement automatique intégré
- Configuration figée à la création du pipeline
- Supporte uniquement des workflows simples

Déploiements basés sur l'intégration git

Avantages

- Simple à créer/utiliser (interface graphique)
- Force l'utilisation d'un repo git

Inconvénients

- Nécessite workspace Fabric ou PPU (pas de pro)
- Ne gère pas nativement l'écrasement des paramètres selon l'environnement
- Pas d'historique des déploiements natif (seulement la date de la dernière synchronisation)
- Besoin d'accès dans le workspace cible pour déployer
- Instable (conflits fréquents, synchronisation compliquée en particulier pour les Warehouses)

fabric-cicd

Avantages

- Permet l'écrasement des paramètres sur quasiment tous les types d'objet Fabric via des configuration puissantes et flexibles
- Fonctionne sur tous les types de workspace (Pro inclus)

Inconvénients

- Pas d'historique de déploiement natif (compensé par les logs Azure DevOps / GitHub)
- Besoin d'accès dans le workspace cible pour déployer (compensé par l'utilisation de Service Principals)
- Plus complexe à utiliser

02

La CI/CD dans Fabric avec fabric-cicd

Deep dive sur la librairie Python fabric-cicd



Notre choix : fabric-cicd

Approche code-first

Possibilité de maîtriser un maximum d'éléments de déploiement (paramétrisation fine, utilisation de regex, ordre de déploiement des workspaces...).

Configuration par environnement

Ex : aucune suppression automatique (unpublish) en prod.

Librairie supportée par Microsoft

Des fonctionnalités sont ajoutées fréquemment (la v1.1.0 date du 27 mai 2026), ça nous évite de manipuler directement les APIs Fabric.

`</>` **fabric-cicd** 1.1.0 latest ▾

Méthodes de déploiement

Impérative : `publish_all_items()` et `unpublish_all_orphan_items()`

Publie (ou supprime) les items d'un dossier (filtre par type possible) dans le workspace (identifié par son *workspace_id*).
Utilise le fichier *parameter.yml* pour modifier le code lors du déploiement.

Déclarative : `deploy_with_config()`

Déploiement piloté par un fichier *config.yml*.
Utilise le fichier *parameter.yml* pour modifier le code lors du déploiement.

```
workspace = FabricWorkspace(  
    workspace_id="your-workspace-id",  
    repository_directory="/path/to/repo",  
    item_type_in_scope=["Notebook", "DataPipeline"],  
    token_credential=token_credential, # or any other TokenCredential  
)  
  
changed = get_changed_items(workspace.repository_directory)  
  
if changed:  
    publish_all_items(workspace, items_to_include=changed)  
else:  
    print("No changed items detected - skipping deployment.")  
  
# Deploy using a config file  
deploy_with_config(  
    config_file_path="C:/dev/workspace/config.yml", # required  
    token_credential=AzureCliCredential(), # required  
    environment="dev"  
)
```


Fichier config.yml

Un fichier par dossier dans la solution du repo

Pilote le déploiement du dossier dans chaque environnement du workspace cible (ex : Demo GUSS – Reporting [dev/tst/prd]) :

1. Nom de chaque workspace
2. Ordre de déploiement par rapport aux autres workspaces
3. Filtre sur les types/noms d'items à déployer
4. Exclusion possible par regex
5. Différentiation du comportement en fonction de l'environnement (publish/unpublish)

```
core:
  workspace:
    dev: "Demo GUSS - Reporting [dev]"
    test: "Demo GUSS - Reporting [tst]"
    prod: "Demo GUSS - Reporting [prd]"

  deployment_order: 3
  repository_directory: "." # relative path from config.yml location

  item_types_in_scope:
    - SemanticModel
    - Report
    - GraphQLApi

  parameter: "parameter.yml" # relative path

publish:
  # Don't publish items matching this pattern
  exclude_regex: "^DONT_DEPLOY.*"

  folder_exclude_regex: "^DONT_DEPLOY_FOLDER/"

  skip:
    dev: false
    test: false
    prod: false
```

Fichier parameter.yml

Un fichier par dossier dans la solution du repo

Remplace des éléments du code avant le déploiement :

- Par défaut, s'applique à tous les fichiers du dossier
- Filtre possible par type/nom/propriété JSON
- Utilisation de regex possible
- Peut également lier la source de donnée d'un modèle sémantique à une connexion Fabric (*semantic_model_binding*)

```
find_replace:
  - find_value: "74a54798-8c0c-4a37-a67f-0c36da528af1" # storage workspace id
    replace_value:
      dev: "$workspace.Demo GUSS - Storage [dev]"
      test: "$workspace.Demo GUSS - Storage [tst]"
      prod: "$workspace.Demo GUSS - Storage [prd]"
    item_type: "GraphQLApi" # filter on API for GraphQL files
    item_name: "GQL_QueryWarehouseData" # filter on specific API for GraphQL files

  - find_value: "830dbdef-05e8-4c09-99c7-7371eff15e10" # warehouse id
    replace_value:
      dev: "$workspace.Demo GUSS - Storage [dev].$items.Warehouse.WH_LRDLG.$id"
      test: "$workspace.Demo GUSS - Storage [tst].$items.Warehouse.WH_LRDLG.$id"
      prod: "$workspace.Demo GUSS - Storage [prd].$items.Warehouse.WH_LRDLG.$id"
    item_type: "GraphQLApi" # filter on API for GraphQL files
    item_name: "GQL_QueryWarehouseData" # filter on specific API for GraphQL files

semantic_model_binding:
  models:
    - semantic_model_name: "Demo Report 1"
      connection_id:
        dev: "e18ac53d-5c43-4e83-8f78-ee41119c5430"
        test: "499a76df-6de8-429c-9a27-01a28ca20e59"
        prod: "5a92bb7e-9524-43eb-abdb-0003627a9892"
```

Limitations

Les principales limitations connues (v1.0.0)

- Le déploiement des connexions n'est pas supporté (à créer manuellement via l'interface ou les APIs Fabric)
- Le *find_replace* ne s'applique pas sur les items du type : Activator, Environment, Eventhouse, KQL Database, Lakehouse, ML Experiment, Mounted Data Factory, SQL Database, Warehouse
- Le *semantic_model_binding* ne s'applique que sur une seule source de données du modèle sémantique (les sources de données additionnelles doivent être configurées manuellement après le déploiement)
- Le contenu d'une SQL Database ou d'un Warehouse n'est pas déployé (à déployer manuellement avec un fichier DACPAC par exemple)



DÉMO

Déploiement avec fabric-cicd

MERCI !

Des questions ?

[✉ email] • [ LinkedIn] • [@Twitter]

[Lien vers les ressources partagées]

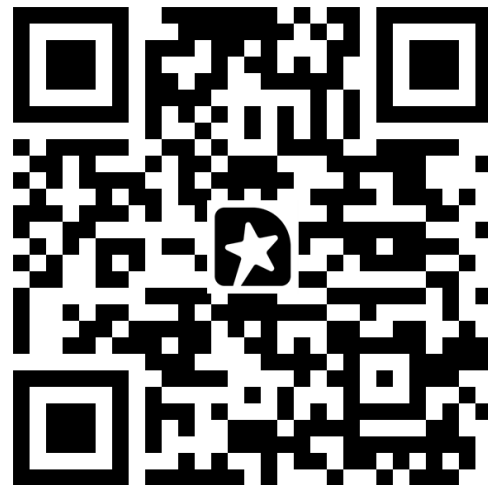


Votre avis compte.

Cette session était la vôtre.
Aidez-nous à construire la prochaine.

2 minutes suffisent pour nous dire ce qui vous a marqué,
ce que vous souhaitez approfondir, et ce que nous pouvons améliorer.
Chaque retour est lu et pris en compte.

Merci de votre participation 🙏



Automatiser vos déploiements Fabric avec fabric-cicd